BHARATI VIDYAPEETH'S COLLEGE OF ENGINEERING, LAVALE, PUNE

DEPARTMENT OF COMPUTER ENGINEERING

DATA STRUCTURE MINI PROJECT

IMPLEMENTATION OF KEYWORD FREQUENCY COUNTER

[Hash Table | Python]

Submitted by	RUDRAKSHI KOUL
Roll Number	09
Class / Semester	SE-B COMP | 2026–27 | Semester I
Subject	Data Structures (Python)
Academic Year	2026–2027

Submitted to the Department of Computer Engineering

BHARATI VIDYAPEETH'S COLLEGE OF ENGINEERING, LAVALE, PUNE

DEPARTMENT OF COMPUTER ENGINEERING

CERTIFICATE

This is to certify that the mini project report submitted by Rudrakshi Koul (Roll Number 09) on "Implementation of Keyword Frequency Counter [HashTable]" is the work completed as part of the Data Structures practical course during the academic year 2026–2027, in partial fulfillment of the requirements for the Bachelor of Engineering in Computer Engineering at Bharati Vidyapeeth's College of Engineering, Lavale, Pune.

Subject Teacher **Head of Department** **Principal**

Date: _____

Place: Lavale, Pune

Acknowledgement

I would like to thank my subject teacher for guiding me throughout the Data Structures course and for the opportunity to work on this mini project. I am also grateful to the Department of Computer Engineering at Bharati Vidyapeeth's College of Engineering, Lavale, for providing the facilities needed to complete it. Finally, I thank my friends and family for their constant support and encouragement.

Rudrakshi Koul
Roll Number 09
SE-B COMP

Contents

1. Abstract
2. Problem Statement
3. Objectives
4. Theory
5. Requirements
6. Algorithm
7. Flowchart / Program Design
8. Python Program
9. Sample Output
10. Test Cases
11. Advantages
12. Limitations
13. Conclusion

1. Abstract

A Keyword Frequency Counter counts how many times each word occurs in a given text. This project uses Python's dictionary as a hash-table-based structure to store each normalized word as a key and its count as the value. The program cleans punctuation, converts text to lowercase, counts words, and displays the frequency table in descending order of frequency.

2. Problem Statement

Develop a Python program that accepts a text input, identifies individual words, and calculates the frequency of every unique word using a hash table. The output should clearly show each word and its number of occurrences.

3. Objectives

- Understand hashing and key-value storage.
- Use a dictionary to count word occurrences efficiently.
- Normalize text by converting it to lowercase and removing punctuation.
- Display unique words with their frequencies.
- Evaluate the algorithm using sample inputs and test cases.

4. Theory

4.1 Hash Table

A hash table stores data as key-value pairs. A hash function maps a key to a location where its value can be stored or retrieved. Average lookup and insertion are typically O(1), although collisions and implementation details can affect performance.

4.2 Python Dictionary

Python's dict is a built-in mapping type implemented using hashing. In this project, each unique word is a key and the number of occurrences is its value. When a word is seen for the first time, its count is initialized to 1; later occurrences increment the existing count.

4.3 Text Normalization

The program converts text to lowercase so that words such as 'Data' and 'data' are counted together. It uses a regular expression to extract word-like tokens while ignoring most punctuation.

Operation	Average Time Complexity	Purpose
Tokenize text	O(n)	Extract words from text
Update word count	O(1) per word	Insert or increment dictionary key
Count all words	O(n) average	Process all tokens
Sort results	O(k log k)	Order unique words by frequency

Here, n is the number of characters/tokens processed and k is the number of unique words.

5. Requirements

- Hardware: Computer/laptop.
- Software: Python 3.x and an editor or IDE.

- Concepts: Dictionaries/hash tables, strings, regular expressions, loops, and sorting.

6. Algorithm

1. Accept a paragraph or sentence from the user.
2. Convert the input to lowercase.
3. Extract word tokens using a regular expression.
4. Create an empty dictionary named frequency.
5. For each token, increase its count if it already exists; otherwise initialize its count to 1.
6. Sort the dictionary entries by frequency in descending order, then alphabetically for ties.
7. Display the total number of words and each unique word with its frequency.

7. Flowchart / Program Design

START → Read text → Convert to lowercase → Extract word tokens

For each word: if key exists, increment count; otherwise set count = 1

Sort frequency pairs → Display total and word counts → STOP

8. Python Program

```
import re
from collections import defaultdict


def count_keywords(text):
    # Extract alphabetic words and apostrophe-containing words
    words = re.findall(r"[a-zA-Z]+(?:'[a-zA-Z]+)?", text.lower())
    frequency = defaultdict(int)

    for word in words:
        frequency[word] += 1

    # Highest frequency first; alphabetical order breaks ties
    sorted_frequency = sorted(
        frequency.items(),
        key=lambda item: (-item[1], item[0])
    )
    return words, sorted_frequency


def main():
    text = input("Enter text: ").strip()

    if not text:
        print("No text entered.")
        return

    words, frequency = count_keywords(text)
    print("\nTotal words:", len(words))
    print("Unique words:", len(frequency))
    print("\nKeyword Frequency")
    print("-----------------")

    for word, count in frequency:
        print(f"{word:<15} {count}")

if __name__ == "__main__":
```

```
main()
```

9. Sample Output

```
Enter text: Data structures are useful. Data helps solve problems.

Total words: 8
Unique words: 7

Keyword Frequency
-----------------
data            2
are             1
helps           1
problems        1
solve           1
structures      1
useful          1
```

10. Test Cases

Test	Input	Expected Result	Status
1	Data data DATA	data: 3	Pass
2	Hi, hi! Hello.	hi: 2, hello: 1	Pass
3	One two three	Each word has count 1	Pass
4	Empty input	No text entered.	Pass
5	A a, B b.	a: 2, b: 2	Pass

11. Advantages

- Dictionary-based counting is concise and efficient.
- Average O(1) dictionary updates make counting scalable for ordinary text.
- Case normalization combines differently capitalized versions of a word.
- Results are easy to read and can be extended for larger text-analysis tasks.

12. Limitations

- The tokenization rule is basic and may not handle every language or special word format.
- The program counts words, not meaning or context.
- Sorting adds O(k log k) time when there are k unique words.

13. Conclusion

The project demonstrates how a hash-table-based dictionary can count word occurrences efficiently. Text normalization makes the results consistent, and sorting provides a clear frequency report. The project reinforces hashing, key-value mappings, string processing, and basic algorithm analysis.